TOPIK 4

Merancang, Mengimplementasikan, dan Mengevaluasi Pembelajaran Berdiferensiasi

(Lembar Kerja Kelompok 4.C. Rancangan Pembelajaran Berdiferensiasi dengan pendekatan Understanding by Design (UbD))

Diajukan untuk Memenuhi Tugas Pembelajaran Berdiferensiasi

Disusun Oleh:

Ahmad Nursyifa
Anthonio Akbar
Andi Dwi Irfandi
Aldrian Rizki Kusuma
Dian Budi Elnursa

PROGRAM STUDI INFORMATIKA

PENDIDIKAN PROFESI GURU

2026

PERENCANAAN PEMBELAJARAN MENDALAM

MATERI PERCABANGAN

Satuan Pendidikan **:** SMAN 15 Bandung.
Mata Pelajaran **:** Informatika
Fase/Kelas/Semester : F/XI/2
Alokasi Waktu **:** 1 Pertemuan (1x2JP)

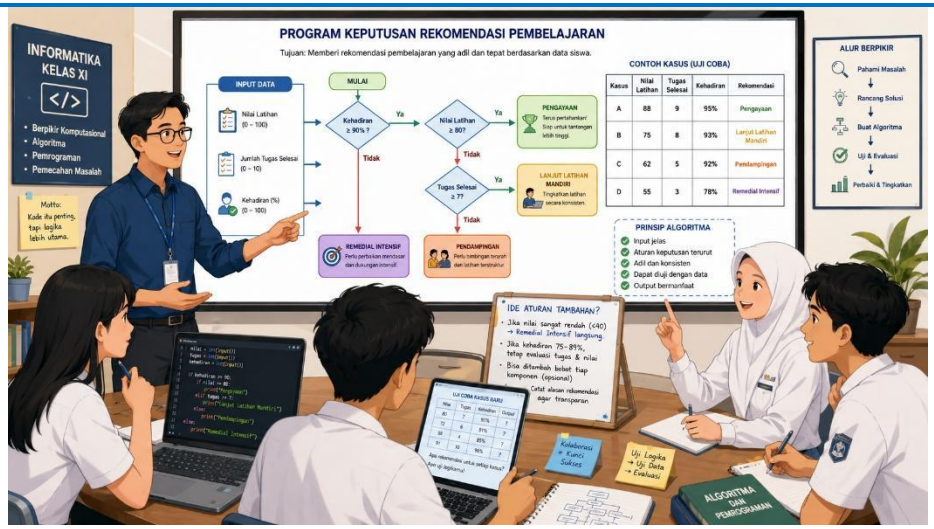
Identifikasi	**Kesiapan Murid :** **1. Pengetahuan Awal** a. Murid telah mengenal konsep algoritma dasar, urutan instruksi, variabel, tipe data sederhana, operator aritmetika, operator perbandingan, dan input-output dasar. b. Murid memiliki kemampuan digital yang beragam; sebagian lancar menggunakan laptop/IDE, sedangkan sebagian lain masih membutuhkan pendampingan teknis ketika menulis dan menjalankan kode. c. Murid membutuhkan contoh kontekstual agar memahami bahwa percabangan bukan sekadar sintaks, tetapi cara program mengambil keputusan berdasarkan kondisi tertentu. d. Perlu disiapkan *scaffolding* berupa contoh kode sederhana, kartu operator logika, template pseudocode, dan *test case* agar semua murid dapat mengikuti aktivitas secara bertahap. **2. Minat dan Motivasi Belajar** a. Murid cenderung lebih antusias ketika masalah pemrograman dikaitkan dengan kehidupan sekolah b. Aktivitas berkelompok dapat meningkatkan motivasi karena murid dapat berbagi peran sebagai analis masalah, penulis algoritma, programmer, penguji, dan *presenter*. c. Pembelajaran dirancang agar murid tidak hanya menyalin kode, tetapi mengalami proses berpikir, membuat keputusan, menguji, keliru, memperbaiki, dan menjelaskan alasan logisnya.

	3. **Latar Belakang Murid** a. Murid kelas XI berada pada fase perkembangan berpikir yang mulai mampu menganalisis kondisi majemuk, membandingkan alternatif, serta menyusun argumen berdasarkan kriteria tertentu. b. Sebagian murid lebih nyaman belajar melalui visualisasi alur (*flowchart*), sebagian melalui contoh kode langsung, dan sebagian lain melalui diskusi atau simulasi peran. c. Keterampilan komunikasi dan kolaborasi perlu dilatih karena pemrograman dalam konteks nyata tidak hanya menuntut jawaban benar, tetapi juga kejelasan alur, dokumentasi, dan kemampuan menjelaskan solusi. 4. **Kebutuhan Belajar** a. Pendekatan kontekstual: masalah dirancang dekat dengan situasi sekolah agar murid memahami fungsi percabangan dalam pengambilan keputusan. b. *Scaffolding* bertahap: dari membaca masalah, menentukan *input-output*, menyusun kondisi, menulis *pseudocode*, mengimplementasikan kode, hingga menguji program. c. Diferensiasi: disediakan pilihan tingkat tantangan, pilihan bentuk produk awal, serta bantuan teknis yang berbeda sesuai kesiapan murid. d. Umpan balik langsung: guru memberi pertanyaan penuntun ketika murid mengalami kesalahan logika, bukan langsung memberikan jawaban akhir.
	Karakteristik Mata Pelajaran : 1. Informatika menekankan kemampuan berpikir komputasional, yaitu memecahkan masalah secara sistematis, logis, efisien, dan dapat diimplementasikan oleh komputer. 2. Materi percabangan memiliki karakteristik berbasis logika keputusan: program tidak berjalan secara linear semata, tetapi memilih aksi berdasarkan kondisi benar atau salah. 3. Pembelajaran percabangan perlu memadukan pemahaman konsep, representasi algoritmik, sintaks bahasa pemrograman, pengujian *test case*, dan refleksi kualitas solusi. 4. Kesalahan yang sering muncul bukan hanya kesalahan sintaks, tetapi juga kesalahan logika, seperti batas nilai yang tumpang tindih, kondisi tidak lengkap, atau urutan pengecekan yang kurang tepat.

	Dimensi Profil Lulusan : 1. **Penalaran kritis**: murid menganalisis masalah, menentukan kondisi, membandingkan alternatif keputusan, dan mengevaluasi hasil program berdasarkan *test case*. 2. **Kreativitas**: murid merancang solusi program dengan konteks yang dapat dipilih dan menyesuaikan tampilan *output* agar komunikatif. 3. **Kolaborasi**: murid bekerja dalam kelompok kecil dengan pembagian peran yang jelas dan saling memberi umpan balik. 4. **Komunikasi**: murid menjelaskan logika percabangan, alasan pemilihan kondisi, dan hasil pengujian program secara runtut. 5. **Mandiri dan reflektif**: murid menyadari kesalahan sebagai bagian dari proses debugging dan perbaikan algoritma.
Desain Pembelajaran	**Capaian Pembelajaran :** Murid mampu memahami konsep strategi algoritmik, mengembangkan program komputer terstruktur dalam notasi algoritma atau notasi lain berdasarkan strategi algoritmik yang tepat. Selain itu, peserta didik mampu mengembangkan, melakukan pemeliharaan dan penyempurnaan algoritma standar ke dalam kode sumber program dengan memperhatikan kualitasnya. Murid juga mampu merancang dan mengimplementasi sebuah program yang menggunakan struktur data kompleks dan tepat menggunakan library atau perangkat yang tersedia.
	Tujuan Pembelajaran : TP 1.1: Murid mampu menjelaskan konsep percabangan sebagai strategi algoritmik untuk menyelesaikan masalah yang memiliki beberapa kemungkinan keputusan. TP 1.2: Murid mampu menganalisis kebutuhan input, proses, kondisi, dan *output* pada masalah kontekstual berbasis keputusan. TP 1.3: Murid mampu menyusun algoritma percabangan dalam bentuk *pseudocode* atau *flowchart* secara logis dan terstruktur. TP 1.4: Murid mampu mengimplementasikan algoritma percabangan sederhana ke dalam kode sumber program menggunakan struktur *if*, *if-else*, *else-if*, atau *nested if* sesuai kebutuhan masalah. TP 1.5: Murid mampu menguji program berdasarkan beberapa *test case*.

	Alur Tujuan Pembelajaran (ATP) & Pemetaan Per Pertemuan		
	Pertemuan	**Tujuan Pembelajaran**	**Indikator Pencapaian**
	1 (2JP)	TP 1.1: Murid mampu menjelaskan konsep percabangan sebagai strategi algoritmik untuk menyelesaikan masalah yang memiliki beberapa kemungkinan keputusan. TP 1.2: Murid mampu menganalisis kebutuhan input, proses, kondisi, dan output pada masalah kontekstual berbasis keputusan. TP 1.3: Murid mampu menyusun algoritma percabangan dalam bentuk pseudocode atau flowchart secara logis dan terstruktur. TP 1.4: Murid mampu mengimplementasikan algoritma percabangan sederhana ke dalam kode sumber program menggunakan struktur if, if-else, else-if, atau nested if sesuai kebutuhan masalah. TP 1.5: Murid mampu menguji program berdasarkan beberapa test case.	1. Murid memahami fungsi percabangan dalam program. 2. Murid menganalisis masalah sistem tindak lanjut belajar berbasis kriteria. 3. Murid menyusun algoritma dalam pseudocode/flowchart. 4. Murid mengimplementasikan kode percabangan dan menguji test case. 5. Murid merefleksikan kesalahan logika dan perbaikan solusi.

	Masalah Kontekstual *Problem Based Learning* **Skenario Masalah PBL**		



Guru Informatika ingin membuat program sederhana untuk membantu menentukan tindak lanjut belajar murid setelah latihan pemrograman. Program harus membaca beberapa data, misalnya nilai latihan, jumlah tugas yang sudah selesai, dan kehadiran. Berdasarkan data tersebut, program memberi rekomendasi: Pengayaan, Lanjut Latihan Mandiri, Pendampingan, atau Remedial Intensif. Namun, aturan keputusan harus jelas, adil, tidak tumpang tindih, dan dapat diuji dengan beberapa contoh kasus.

Tantangan Murid

1. Bagaimana mengubah aturan keputusan tersebut menjadi algoritma percabangan yang logis?
2. Kondisi apa saja yang perlu dicek oleh program agar output tidak keliru?
3. Bagaimana memastikan program bekerja benar untuk berbagai kemungkinan input?
4. Bagaimana kelompok menjelaskan alur logika program kepada orang lain?

Pemahaman Bermakna

Percabangan membuat program mampu mengambil keputusan berdasarkan kondisi. Dalam kehidupan nyata, banyak keputusan dibuat dengan membandingkan data terhadap kriteria tertentu. Dengan memahami percabangan, murid belajar bahwa kualitas program tidak

	hanya ditentukan oleh kode yang berjalan, tetapi juga oleh ketepatan logika, kejelasan aturan, dan kemampuan menguji berbagai kemungkinan kasus.
	Pertanyaan Pemantik: 1. Bagaimana komputer dapat membuat keputusan jika komputer hanya mengikuti instruksi yang kita tulis? 2. Mengapa urutan kondisi dalam percabangan dapat memengaruhi hasil program? 3. Apa yang terjadi jika dua kondisi dalam program saling tumpang tindih? 4. Bagaimana cara membuktikan bahwa program yang kita buat sudah mengambil keputusan dengan benar? 5. Dalam konteks sekolah, keputusan apa saja yang dapat dibantu oleh program percabangan sederhana?
	Praktik Pedagogis : 1. Metode Diskusi dan Tanya Jawab 2. Strategi pendukung a. diskusi kelompok b. demonstrasi singkat c. tanya jawab berbasis pertanyaan penuntun, d. debugging terbimbing e. peer feedback f. refleksi akhir. 3. Prinsip pembelajaran mendalam a. berkesadaran (murid sadar tujuan dan proses berpikirnya) b. bermakna (masalah dekat dengan konteks sekolah) c. menggembirakan (aktivitas tantangan, kolaborasi, serta presentasi singkat). 4. Model *Problem Based Learning* (PBL) a. Orientasi Murid pada Masalah b. Mengorganisasi Murid untuk Belajar c. Membimbing Penyelidikan Individu/Kelompok d. Mengembangkan dan Menyajikan Hasil Karya e. Menganalisis dan Mengevaluasi Proses Pemecahan Masalah

	Kemitraan dan Lingkungan Pembelajaran : 1. Kemitraan di kelas dibangun melalui pembagian peran anggota kelompok a. analis masalah b. penulis algoritma c. programmer d. tester e. presenter. 2. Guru berperan sebagai fasilitator yang memberi scaffolding dan umpan balik, bukan sebagai satu-satunya sumber jawaban. 3. Lingkungan pembelajaran dibuat aman untuk mencoba, melakukan kesalahan, bertanya, dan memperbaiki program.
	Sarana, Prasarana, dan Pemanfaatan Digital : 1. Ruang belajar a. Kelas atau laboratorium komputer dengan tata letak yang memungkinkan diskusi kelompok. 2. Perangkat a. Laptop/komputer murid atau perangkat guru yang dapat digunakan bergantian; proyektor; papan tulis/spidol. 3. Aplikasi/Tools a. IDE C++ (Dev-C++) b. *online compiler* (*OnlineGDB, Programiz, Replit*) 4. Bahan ajar a. Slide ringkas konsep percabangan b. LKPD, kartu kondisi c. *template pseudocode* d. contoh kode e. lembar *test case*. 5. Koneksi internet 6. Alternatif teknis a. Jika perangkat terbatas, murid dapat menyusun pseudocode/flowchart secara manual, kemudian satu perangkat digunakan untuk demonstrasi dan pengujian kode.

<table>
<tr><td rowspan="5">Diferensiasi</td><td colspan="2">Rasional Diferensiasi
Diferensiasi diterapkan karena kemampuan awal murid dalam pemrograman biasanya tidak merata. Pada materi percabangan, perbedaan tersebut tampak pada kemampuan membaca masalah, memahami operator logika, menulis sintaks, dan menguji output. Oleh karena itu, guru menyediakan variasi konten, proses, produk, dan dukungan agar semua murid tetap bekerja menuju tujuan inti yang sama, yaitu memahami dan menerapkan logika percabangan.</td></tr>
<tr><td colspan="2">Pemetaan Diferensiasi Konten, Proses, Produk, dan Lingkungan</td></tr>
<tr><td>Jenis Diferensiasi</td><td>Implementasi Konkret</td></tr>
<tr><td>Diferensiasi Konten</td><td>1. Murid pemula memperoleh ringkasan konsep percabangan, contoh if-else sederhana, dan kartu operator perbandingan/logika.
2. Murid sedang memperoleh contoh else-if, tabel aturan keputusan, dan template pseudocode setengah jadi.
3. Murid mahir memperoleh tantangan tambahan berupa nested if, operator logika majemuk, validasi input, atau perbaikan efisiensi kondisi.</td></tr>
<tr><td>Diferensiasi Proses</td><td>1. Kelompok heterogen dibentuk agar murid dapat saling mendukung; guru memberi bantuan berbeda sesuai kebutuhan kelompok.
2. Murid boleh memilih representasi awal: flowchart, pseudocode, atau tabel keputusan sebelum menulis kode.
3. Guru menyediakan checkpoint bertahap: analisis masalah -> kondisi -> algoritma -> kode -> test case -> refleksi.</td></tr>
<tr><td>Diferensiasi Produk</td><td>1. Produk minimum: tabel input-kondisi-output dan pseudocode percabangan yang benar.
2. Produk standar: kode program percabangan yang berjalan dengan minimal 3 test case.
3. Produk pengayaan: program dengan validasi input, kondisi majemuk, tampilan output lebih komunikatif, dan dokumentasi singkat.</td></tr>
</table>

<table>
<tr><td></td><td colspan="2">Diferensiasi Lingkungan Belajar</td><td colspan="2">1. Murid yang membutuhkan fokus dapat bekerja pada area lebih tenang atau menggunakan lembar kerja cetak.
2. Murid yang siap tampil dapat mendemonstrasikan kode melalui proyektor atau berbagi layar.
3. Jika perangkat terbatas, guru menerapkan pair programming atau rotasi peran agar semua murid tetap terlibat.</td></tr>
<tr><td></td><td colspan="4">Diferensiasi Berdasarkan Kesiapan Murid</td></tr>
<tr><td></td><td>Kesiapan</td><td>Ciri Murid</td><td>Tugas Inti</td><td>Bantuan/Pengayaan</td></tr>
<tr><td></td><td>Kelompok Dasar</td><td>Murid masih kesulitan memahami operator perbandingan atau alur if-else.</td><td>Menganalisis satu variabel nilai dan membuat keputusan dua pilihan: lulus/remedial.</td><td>Kartu operator, contoh kode lengkap dengan bagian yang dikosongkan, dan pendampingan guru lebih sering.</td></tr>
<tr><td></td><td>Kelompok Menengah</td><td>Murid memahami if-else, tetapi masih perlu latihan menyusun banyak kondisi.</td><td>Membuat else-if untuk kategori nilai dan rekomendasi tindak lanjut.</td><td>Template tabel keputusan, daftar test case, dan pertanyaan penuntun.</td></tr>
<tr><td></td><td>Kelompok Mahir</td><td>Murid mampu menulis kode dasar dan siap menerima kompleksitas tambahan.</td><td>Menambahkan kondisi majemuk, nested if, validasi rentang input 0-100, atau pengecekan tugas/kehadiran.</td><td>Challenge card, kebebasan mengembangkan fitur, dan peer tutor kepada kelompok lain.</td></tr>
<tr><td></td><td colspan="4">Penerapan Diferensiasi</td></tr>
<tr><td></td><td colspan="4">Skenario singkat penerapan diferensiasi saat microteaching
1. Sebelum kegiatan inti, guru menampilkan asesmen diagnostik cepat 3 pertanyaan untuk memetakan kesiapan murid.
2. Guru membagikan tiga kartu tantangan: Level 1 (if-else sederhana), Level 2 (else-if), Level 3 (nested if/operator logika). Murid memilih level awal, tetapi guru dapat mengarahkan berdasarkan hasil diagnostik.</td></tr>
</table>

<table>
<tr><td></td><td>3. Saat penyelidikan, guru bergerak antar kelompok menggunakan pertanyaan penuntun yang berbeda: kelompok dasar fokus pada benar/salah kondisi, kelompok menengah fokus pada urutan kondisi, kelompok mahir fokus pada kualitas dan pengujian program.
4. Produk akhir tidak harus seragam: semua kelompok menunjukkan alur keputusan, tetapi kedalaman kode dan fitur boleh berbeda sesuai kesiapan.
5. Pada presentasi singkat, guru menekankan bahwa diferensiasi bukan membedakan kualitas murid, melainkan menyediakan jalur belajar yang sesuai agar semua murid mencapai tujuan inti.</td></tr>
<tr><td>Pengalaman Belajar</td><td>Langkah – Langkah Pembelajaran :
Prinsip Pembelajaran Mendalan (Berkesadaran, Bermakna, Menggembirakan)
Pengalaman Belajar (Memahami, Mengaplikasikan, Merefleksikan)

PERTEMUAN 1
<table>
<tr><th rowspan="2">TP</th><th rowspan="2">Kegiatan</th><th colspan="4">Aktivitas Belajar</th><th rowspan="2">Alokasi Waktu</th></tr>
<tr><th>Guru</th><th>Murid</th><th>Sintaks PBL</th><th>Sintaks DL</th></tr>
<tr><td>TP 1.1 - TP 1.5</td><td>Pendahuluan</td><td>● Membuka pembelajaran dengan salam, doa, presensi, dan pengecekan kesiapan perangkat.
● Menyampaikan tujuan pembelajaran dan manfaat percabangan dalam program.
● Mengajukan pertanyaan pemantik: Bagaimana komputer dapat memilih tindakan yang berbeda?</td><td>● Menjawab salam, berdoa, dan menyiapkan perangkat atau LKPD.
● Menyimak tujuan pembelajaran dan menghubungkan dengan pengalaman menggunakan aplikasi digital.</td><td>-</td><td>(Memahami)
Berkesadaran dan Bermakna</td><td>10 Menit</td></tr>
</table></td></tr>
</table>

			• Melaksanakan asesmen diagnostik cepat melalui pertanyaan lisan	• Menjawab pertanyaan pemantik dan asesmen diagnostik sesuai pemahaman awal.			
	TP 1.1, TP 1.2	Inti 1: Orientasi pada Masalah	• Menampilkan skenario masalah: program rekomendasi tindak lanjut belajar setelah latihan pemrograman. • Membacakan contoh aturan awal dan menunjukkan bahwa aturan yang tidak jelas dapat menghasilkan keputusan keliru. • Mengajukan pertanyaan: data apa yang harus menjadi input, kondisi apa yang perlu diperiksa, dan output apa yang diharapkan?	• Membaca skenario dan mengidentifikasi masalah utama. • Menuliskan dugaan input, kondisi, dan output pada LKPD. • Menyampaikan ide awal tentang aturan keputusan yang adil dan tidak tumpang tindih.	*Sintaks 1: Orientasi murid pada masalah*	***(Memahami)*** *Bermakna dan Menggembirakan*	10 Menit
	TP 1.2, TP 1.3	Inti 2: Mengorganisasi Murid	• Membagi murid ke dalam kelompok kecil 3-4 orang secara heterogen.	• Bergabung dengan kelompok dan memilih peran	*Sintaks 2: Mengorganisasi Murid untuk belajar*	***(Memahami)*** *Berkesadaran,*	8 Menit

			• Menjelaskan peran anggota: analis masalah, penulis algoritma, programmer, tester, dan presenter. • Membagikan LKPD, kartu operator logika, dan kartu tantangan diferensiasi Level 1-3. • Menekankan bahwa setiap kelompok boleh memilih representasi awal: tabel keputusan, pseudocode, atau flowchart	sesuai kesepakatan. • Mempelajari LKPD dan memilih level tantangan yang sesuai. • Menyepakati konteks solusi dan bentuk representasi awal.			
	TP 1.2 - TP 1.5	Inti 3: Penyelidikan Kelompok	• Membimbing kelompok menganalisis aturan keputusan dan menyusun tabel input-kondisi-output. • Memberikan scaffolding berbeda: contoh untuk pemula, pertanyaan penuntun untuk kelompok menengah, dan challenge untuk kelompok mahir.	• Menyusun algoritma dalam bentuk tabel keputusan, pseudocode, atau flowchart. • Menulis kode program percabangan pada IDE/lembar kerja sesuai kesiapan kelompok.	*Sintaks 3: Membimbing Penyelidikan individu/kelompok*	***(Mengaplikasikan);*** *Bermakna,, Berkesadaran, Menggembirakan*	32 Menit

			• Mendemonstrasikan singkat struktur if, if-else, else-if, dan nested if sesuai kebutuhan. • Mengamati proses, memberi umpan balik, dan membantu debugging tanpa langsung memberikan jawaban akhir.	• Menguji minimal 3 test case, mencatat hasil, dan memperbaiki kesalahan logika atau sintaks. • Bertanya, berdiskusi, dan saling memberi dukungan dalam kelompok.			
	TP 1.3 - TP 1.5	Inti 4: Menyajikan Hasil	• Meminta beberapa kelompok menampilkan solusi secara singkat: alur keputusan, potongan kode, dan hasil test case. • Memfasilitasi tanya jawab dan peer feedback menggunakan format dua apresiasi dan satu saran perbaikan. • Menegaskan konsep kunci: urutan kondisi, operator logika, batas nilai, dan pentingnya pengujian.	• Mempresentasikan hasil kerja kelompok secara ringkas. • Menjelaskan alasan pemilihan kondisi dan hasil test case. • Memberi tanggapan terhadap solusi kelompok lain secara santun dan berbasis bukti.	*Sintaks 4: : Mengembangkan dan menyajikan hasil karya*	***(Merefleksikan)*** *Bermakna dan Menggembirakan*	15 Menit

	TP 1.1 - TP 1.5	Penutup	• Mengajak murid menyimpulkan fungsi percabangan dan langkah menyusun program berbasis kondisi. • Melaksanakan asesmen sumatif singkat/exit ticket individual. • Memberikan apresiasi, tindak lanjut remedial/pengayaan, dan refleksi akhir. • Menutup pembelajaran dengan doa dan salam.	• Menyampaikan kesimpulan pribadi tentang percabangan. • Mengerjakan exit ticket atau post-test singkat secara individual. • Mengisi refleksi murid dan menyampaikan kendala yang masih dirasakan. • Berdoa dan menjawab salam.	*Sintaks 5: Menganalisis dan mengevaluasi proses pemecahan masalah*	***(Merefleksikan)*** *Berkesadaran, Bermakna*	15 Menit
Asesmen Pembelajaran	**Asesmen Awal Pembelajaran** **1. Tujuan Asesmen:** a. Memetakan pemahaman awal murid tentang algoritma, operator perbandingan, operator logika, dan struktur percabangan. b. Memetakan kesiapan teknis murid dalam menggunakan IDE atau online compiler. c. Menjadi dasar guru dalam menentukan kelompok, level tantangan, dan bentuk scaffolding. **2. Jenis dan Bentuk Instrumen:** a. pertanyaan singkat lisan/tulis, polling cepat, atau kartu respons 3 menit. **3. Instrumen Asesmen:** a. Kognitif 1: Apa fungsi algoritma dalam pemrograman? b. Kognitif 2: Apa arti kondisi nilai >= 75 dalam program? c. Kognitif 3: Jika nilai = 80, apakah kondisi nilai >= 85 bernilai benar atau salah? Jelaskan singkat.						

d. Kognitif 4: Apa perbedaan operator = dan == dalam pemrograman C++?
e. Kognitif 5: Sebutkan satu contoh keputusan sehari-hari yang dapat ditulis dengan pola jika... maka...
f. Non-kognitif 1: Seberapa percaya diri kamu menulis kode sederhana? (1-4)
g. Non-kognitif 2: Media belajar mana yang paling membantumu: contoh kode, flowchart, diskusi, atau praktik langsung?
h. Non-kognitif 3: Apakah kamu lebih nyaman bekerja individu, berpasangan, atau kelompok kecil? Mengapa?
i. Non-kognitif 4: Kesulitan apa yang biasanya kamu alami saat belajar pemrograman?

4. Pemanfaatan Hasil Diagnostik:

Kategori	Ciri Jawaban	Tindak Lanjut
Siap dasar	Jawaban menunjukkan belum memahami operator/perbandingan.	Diberi contoh konkret, kartu operator, dan tugas Level 1.
Siap sedang	Memahami operator, tetapi belum lancar menyusun kondisi majemuk.	Diberi tabel keputusan dan tugas Level 2.
Siap mahir	Memahami konsep dan siap menulis kode mandiri.	Diberi challenge Level 3 dan peran peer tutor.

Asesmen Proses Pembelajaran

Aspek	Indikator Formatif	Teknik Pengumpulan Bukti
Analisis masalah	Murid mampu menentukan input, kondisi, dan output dari skenario.	Catatan observasi saat diskusi kelompok.
Algoritma	Murid menyusun pseudocode/flowchart yang urut dan tidak tumpang tindih.	Cek LKPD bagian algoritma.
Implementasi	Murid menulis struktur if/else-if/nested if sesuai kebutuhan.	Cek kode/lembar kerja.
Debugging	Murid menggunakan test case dan memperbaiki kesalahan logika/sintaks.	Cek tabel pengujian.

Kolaborasi	Murid menjalankan peran, berdiskusi, dan menghargai masukan teman.	Observasi sikap dan komunikasi.

Instrumen LKPD Formatif

Bagian LKPD	Fokus	Aktivitas Murid
Bagian A	Analisis masalah	Murid menuliskan data input, kondisi, output, dan risiko kesalahan aturan.
Bagian B	Tabel keputusan	Murid mengisi tabel kondisi dan rekomendasi agar aturan tidak tumpang tindih.
Bagian C	Algoritma	Murid memilih pseudocode atau flowchart sesuai profil belajar.
Bagian D	Implementasi	Murid menulis kode percabangan sesuai level tantangan.
Bagian E	Pengujian	Murid menguji minimal 3 test case dan mencatat perbaikan.

Asesmen Akhir Pembelajaran

1. **Tujuan Asesmen:**
   a. Mengukur pencapaian individu terhadap konsep percabangan dan penerapannya dalam algoritma/kode.
   b. Menilai kemampuan murid membaca kondisi, memprediksi output, dan memperbaiki kesalahan logika sederhana.
   c. Menjadi dasar tindak lanjut pengayaan atau remedial setelah pembelajaran.
2. **Jenis dan Bentuk Instrumen:**
   a. Mengukur pencapaian individu terhadap konsep percabangan dan penerapannya dalam algoritma/kode.
   b. Menilai kemampuan murid membaca kondisi, memprediksi output, dan memperbaiki kesalahan logika sederhana.
   c. Menjadi dasar tindak lanjut pengayaan atau remedial setelah pembelajaran.
3. **Instrumen Asesmen:**

No	Pertanyaan	Pilihan Jawaban / Uraian	Kunci
1	Struktur percabangan digunakan ketika program perlu...	A. mengulang instruksi berkali-kali; B. memilih aksi berdasarkan kondisi; C. menyimpan banyak data; D. menampilkan teks saja	B
2	Jika nilai = 78, kondisi nilai >= 75 bernilai...	A. benar; B. salah; C. error; D. tidak dapat ditentukan	A
3	Operator untuk membandingkan dua nilai apakah sama dalam C++ adalah...	A. =; B. ==; C. !=; D. >=	B
4	Bentuk percabangan yang tepat untuk memilih lebih dari dua kategori nilai adalah...	A. cout; B. cin; C. if-else-if; D. return	C
5	Jika kondisi pertama dalam if bernilai benar, maka pada struktur if-else-if program akan...	A. tetap memeriksa semua kondisi; B. menjalankan blok kondisi benar dan melewati cabang berikutnya; C. berhenti total tanpa output; D. selalu menjalankan else	B
6	Kondisi majemuk nilai >= 75 && tugas >= 3 berarti...	A. salah satu syarat cukup; B. kedua syarat harus terpenuhi; C. nilai harus kurang dari 75; D. tugas tidak diperiksa	B
7	Kesalahan logika yang sering terjadi pada kategori nilai adalah...	A. menggunakan komentar; B. batas nilai tumpang tindih atau tidak lengkap; C. memakai variabel; D. menulis cout	B
8	Mengapa test case diperlukan?	A. agar kode terlihat panjang; B. untuk membuktikan program berjalan benar pada beberapa kondisi; C. untuk mengganti algoritma; D. agar tidak perlu debugging	B
9	Uraian	Tuliskan pseudocode sederhana untuk menentukan 'Lulus' jika nilai >= 75 dan 'Remedial' jika nilai < 75.	Contoh: IF nilai >= 75 THEN

					tampilkan 'Lulus' ELSE tampilkan 'Remedial'.	
		10	Uraian	Sebutkan satu test case untuk program rekomendasi tindak lanjut belajar dan jelaskan output yang diharapkan.	Contoh: nilai 90, tugas 5 -> Pengayaan; alasan memenuhi nilai >= 85 dan tugas >= 4.	

KRITERIA KETERCAPAIAN DAN RUBRIK PENILAIAN

KKTP, Rubrik Produk, Kolaborasi, dan Presentasi

Penilaian

Kriteria Ketercapaian Tujuan Pembelajaran

Rentang Nilai	Kategori	Deskripsi Ketercapaian
86-100	**Sangat Berkembang**	**Murid mampu menjelaskan konsep, menyusun algoritma, menulis kode percabangan yang berjalan, menguji minimal 3 test case, dan menjelaskan perbaikan logika secara mandiri.**
76-85	**Berkembang Sesuai Harapan**	**Murid mampu menyusun algoritma dan kode yang berjalan dengan sedikit kesalahan, serta melakukan pengujian dengan bantuan ringan.**
60-75	**Mulai Berkembang**	**Murid memahami konsep dasar dan dapat menyusun sebagian algoritma/kode, tetapi masih membutuhkan bantuan pada urutan kondisi atau debugging.**
< 60	**Perlu Bimbingan**	**Murid belum menunjukkan pemahaman memadai tentang kondisi dan struktur percabangan; perlu remedial melalui contoh konkret dan latihan bertahap.**

Rubrik Produk Algoritma dan Program

Aspek	4 - Sangat Baik	3 - Baik	2 - Cukup	1 - Perlu Bimbingan
Analisis masalah	Input, kondisi, output lengkap dan relevan; aturan tidak tumpang tindih.	Input, kondisi, output cukup lengkap; ada sedikit bagian yang perlu diperjelas.	Analisis masih parsial; beberapa kondisi kurang tepat.	Belum mampu menentukan input, kondisi, dan output secara benar.
Algoritma	Pseudocode/flowchart runtut, efisien, dan sesuai masalah.	Algoritma runtut dan sesuai masalah dengan sedikit kekurangan.	Algoritma belum runtut atau beberapa keputusan belum tepat.	Algoritma tidak menggambarkan solusi masalah.
Kode program	Kode berjalan, sintaks benar, kondisi tepat, dan output komunikatif.	Kode berjalan dengan kesalahan kecil yang tidak mengganggu hasil utama.	Kode sebagian berjalan, tetapi masih ada kesalahan sintaks/logika.	Kode belum berjalan atau tidak sesuai algoritma.
Pengujian	Minimal 3 test case beragam, output sesuai, dan ada catatan perbaikan.	Ada test case dan output cukup sesuai, tetapi variasi belum lengkap.	Pengujian terbatas dan belum menunjukkan semua kemungkinan kondisi.	Tidak melakukan pengujian atau tidak mencatat hasil.
Kualitas solusi	Solusi jelas, adil, mudah dipahami, dan dapat dikembangkan.	Solusi cukup jelas dan dapat digunakan.	Solusi masih membingungkan pada beberapa bagian.	Solusi belum menunjukkan logika keputusan yang tepat.

Rubrik Kolaborasi dan Presentasi Singkat

Aspek	4	3	2	1
Partisipasi	Aktif menjalankan peran dan membantu kelompok.	Menjalankan peran dengan cukup baik.	Kadang terlibat, tetapi masih pasif.	Tidak terlibat aktif dalam kelompok.
Komunikasi	Menyampaikan ide jelas dan menghargai pendapat teman.	Berkomunikasi cukup jelas dan santun.	Komunikasi belum konsisten atau kurang mendengarkan.	Menghambat komunikasi kelompok.
Tanggung jawab	Menyelesaikan tugas tepat waktu dan berkontribusi nyata.	Menyelesaikan sebagian besar tugas.	Perlu sering diingatkan untuk menyelesaikan tugas.	Tidak menyelesaikan tanggung jawab.
Refleksi	Mampu menjelaskan kesalahan dan perbaikan kelompok.	Mampu menyebutkan sebagian perbaikan.	Refleksi masih umum dan belum spesifik.	Belum mampu merefleksikan proses.

Bobot Penilaian yang Disarankan

Komponen	Bobot	Keterangan
Asesmen diagnostik	Tidak diberi nilai akhir	Digunakan untuk pemetaan kesiapan dan diferensiasi.
Formatif proses/LKPD	30%	Menilai analisis masalah, algoritma, dan proses debugging.
Produk program/solusi	40%	Menilai kualitas kode, hasil test case, dan kesesuaian solusi.
Sumatif individu	20%	Menilai pemahaman konsep dan prediksi output.
Kolaborasi/presentasi	10%	Menilai komunikasi, tanggung jawab, dan refleksi kelompok.

LAMPIRAN LKPD

LKPD Problem Based Learning: Percabangan

LKPD

LKPD Bagian 1

Judul LKPD: Merancang Program Rekomendasi Tindak Lanjut Belajar

Nama Kelompok: ..... Kelas: ..... Anggota: .....

A. Bacalah Masalah

Guru Informatika membutuhkan program sederhana untuk memberikan rekomendasi tindak lanjut belajar setelah murid mengerjakan latihan pemrograman. Program menerima input nilai latihan dan jumlah tugas selesai. Program harus memberikan rekomendasi yang tepat berdasarkan aturan yang disepakati.

B. Analisis Masalah

Komponen	Jawaban Kelompok
Input yang diperlukan	.....
Output yang diharapkan	.....
Kondisi/aturan keputusan	.....
Risiko kesalahan logika	.....

C. Pilih Level Tantangan

Level	Tantangan
Level 1	**Gunakan satu variabel nilai dan if-else: Lulus/Remedial.**
Level 2	**Gunakan else-if untuk kategori: Pengayaan, Lanjut, Pendampingan, Remedial.**
Level 3	**Gunakan nilai + tugas selesai, operator logika, validasi input, atau nested if.**

LKPD Bagian 2

D. Tabel Keputusan

No	Kondisi	Output
1	.....	.....
2	.....	.....
3	.....	.....
4	.....	.....

E. Pseudocode/Flowchart

```
START
INPUT nilai, tugas
IF ..... THEN
        OUTPUT .....
ELSE IF ..... THEN
        OUTPUT .....
ELSE
        OUTPUT .....
END IF
END
```

LKPD Bagian 3

F. Implementasi Kode

Tuliskan kode program kalian pada IDE atau lembar berikut. Pastikan kode menggunakan percabangan sesuai level tantangan.

```
// Tulis kode program di sini
```

G. Pengujian Test Case

No	Nilai	Tugas	Output Diharapkan	Output Aktual	Sesuai?
1	.....	.....	.....	.....	Ya/Tidak
2	.....	.....	.....	.....	Ya/Tidak
3	.....	.....	.....	.....	Ya/Tidak
4	.....	.....	.....	.....	Ya/Tidak

H. Refleksi Kelompok

- **Kesalahan apa yang paling sulit ditemukan?**

  .....

- **Bagaimana kelompok memperbaiki kesalahan tersebut?**

  .....

- **Apa yang akan kalian tingkatkan jika program dikembangkan lebih lanjut?**

  .....

REFLEKSI DAN TINDAK LANJUT

Refleksi Guru, Refleksi Murid, Remedial, dan Pengayaan

Refleksi	**Refleksi Guru** • **Apakah masalah PBL yang diberikan sudah cukup dekat dengan pengalaman murid dan mampu memantik diskusi?** • **Bagian mana dari proses pembelajaran yang menunjukkan murid benar-benar memahami logika percabangan, bukan sekadar menyalin kode?** • **Apakah strategi diferensiasi membantu murid dengan kesiapan berbeda mencapai tujuan pembelajaran? Bukti apa yang mendukung?** • **Kesalahan logika apa yang paling sering muncul dan bagaimana tindak lanjut pembelajarannya?** • **Apakah alokasi waktu untuk analisis masalah, coding, dan pengujian sudah seimbang?** • **Apa perbaikan yang perlu dilakukan pada LKPD, scaffolding, atau pembagian kelompok untuk pertemuan berikutnya?**
	Refleksi Murid • **Hari ini saya memahami bahwa percabangan digunakan untuk ....** • **Bagian yang paling mudah bagi saya adalah ....** • **Bagian yang paling menantang bagi saya adalah ....** • **Kesalahan yang saya/kelompok saya temukan saat menguji program adalah ....** • **Strategi yang membantu saya belajar hari ini adalah ....** • **Pada pembelajaran berikutnya, saya ingin memperbaiki ....**

Remedial dan Pengayaan

Tindak Lanjut	Kegiatan
Remedial	**Murid yang belum mencapai KKTP mengerjakan latihan bertahap: membaca kondisi, melengkapi pseudocode, memperbaiki kode if-else sederhana, dan menguji 2 test case dengan bimbingan guru.**
Pengayaan	**Murid yang sudah mencapai KKTP mengembangkan program dengan validasi input, nested if, operator logika majemuk, atau membuat versi konteks lain seperti sistem rekomendasi ekstrakurikuler/kantin/perpustakaan.**
Tugas Lanjutan	**Murid menulis catatan reflektif singkat tentang satu kesalahan logika yang pernah muncul dan cara memperbaikinya.**
Umpan Balik	**Guru memberikan komentar spesifik pada logika kondisi, bukan hanya nilai akhir atau tampilan output.**

Catatan: perangkat ini dapat disesuaikan dengan kebijakan sekolah, alokasi JP, bahasa pemrograman yang digunakan, serta karakteristik murid di kelas praktik.